Journal of Electronic Science and Technology 23 (2025) 100328

Contents lists available at ScienceDirect

Journal of Electronic Science and Technology

journal homepage: www.keaipublishing.com/en/journals/journal-of-electronic-science-and-technology/



Hybrid big data optimization based energy-efficient and AI-powered green architecture toward smart cities and 5G-IoT applications

Ihab Nassra *, Juan V. Capella

Department of Information and Communication Technologies, Polytechnic University of Valencia, Valencia, 46022, Spain

ARTICLE INFO

Publishing editor: Yu-Lian He

Keywords:
Big data
Compression ratio
Energy-efficient
Internet of things
Mobile cloud computing
Smart cities

ABSTRACT

The convergence of Internet of things (IoT) and 5G holds immense potential for transforming industries by enabling real-time, massive-scale connectivity and automation. However, the growing number of devices connected to the IoT systems demands a communication network capable of handling vast amounts of data with minimal delay. These generated enormous complex, high-dimensional, high-volume, and high-speed data also brings challenges on its storage, transmission, processing, and energy cost, due to the limited computing capabilities, battery capacity, memory, and energy utilization of current IoT networks. In this paper, a seamless architecture by combining mobile and cloud computing is proposed. It can agilely bargain with 5G-IoT devices, sensor nodes, and mobile computing in a distributed manner, enabling minimized energy cost, high interoperability, and high scalability as well as overcoming the memory constraints. An artificial intelligence (AI)-powered green and energy-efficient architecture is then proposed for 5G-IoT systems and sustainable smart cities. The experimental results reveal that the proposed approach dramatically reduces the transmitted data volume and power consumption and yields superior results regarding interoperability, compression ratio, and energy saving. This is especially critical in enabling the deployment of 5G and even 6G wireless systems for smart cities.

1. Introduction

The emergence of 5G has facilitated the expansion of Internet of things (IoT), attributing to its enhanced mobile broadband (eMBB), ultra-reliable low-latency communications (URLLC), and massive machine-type communications (mMTC) [1]. Currently, it has revolutionized the way we interact with the world by enabling seamless connectivity between devices and the Internet. In IoT, millions of devices, such as sensors and actuators, are connected to the Internet via a wireless sensor network (WSN). As stated by market analysts, about 25 billion sensor-enabled devices were deployed by 2020 [1]. Similarly, by 2025, the demand for such devices is estimated to be about 2.1 trillion [2]. However, the proliferation of IoT devices has also led to significant energy cost and massive data generation. The traditional efforts mainly focus on delivering seamless and automated services by advancing wireless sensors in IoT applications [3].

WSNs and cloud computing are two powerful technologies that often complement each other in various applications, particularly in the realm of IoT. By delivering computing services over the Internet, cloud computing exhibits numerous benefits, including cost

* Corresponding author.
E-mail addresses: ihnas@doctor.upv.es (I. Nassra), jcapella@disca.upv.es (J.V. Capella).

https://doi.org/10.1016/j.jnlest.2025.100328
Received 9 November 2024; Received in revised form 16 August 2025; Accepted 26 August 2025
Available online 28 August 2025
1674-862X/© 2025 The Authors. Publishing services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

savings, scalability, flexibility, and accessibility. It can offer scalable storage and processing for more complex and resource-intensive operations. However, this usually needs integrating with edge computing to create a robust and flexible computing infrastructure. Because edge computing performs computational tasks on devices or local servers situated closer to the data source, handling real-time and latency-sensitive tasks locally [4]. Compared with individual cloud computing, such hybrid architecture could significantly reduce latency, enhance processing efficiency, and conserve the bandwidth by filtering and aggregating data locally before it is transmitted to the cloud. It is also beneficial to minimize the transmission of unnecessary data, alleviate network congestion, reduce energy cost, and realize real-time decision-making in resource-constrained environments such as sensor nodes [5].

The exponential growth of Internet-based and distributed computing paradigms, including mobile computing, cloud computing, and IoT, has triggered an unprecedented increase in data generation [6]. The generated high-volume, high-speed, and high-dimensional data, often characterized by strong inter-feature correlations and significant redundancy, poses huge challenges on real-time processing, storage, and analysis. Despite 5G-enabled technologies having significantly improved data generation and transmission rates, they also intensify the need for scalable, efficient, and intelligent data management and optimization techniques. Addressing these challenges has become a pivotal priority to exploiting the full potential of IoT-enabled intelligent systems. Data optimization, referring to improving the data collection, transmission, storage, processing, and analysis, is especially crucial for this kind of IoT systems [7]. Considering excessive energy cost in wireless transmission, energy-saving in WSN is also critical. Obviously, reducing the amount of bits to be transmitted is a straightforward but valid way to minimize this energy cost, such as data compression and optimization. However, most energy-saving mechanisms are not applicable to sensor nodes, mobile computing, and edge computing, due to the low energy utilization and operation speed as well as limited scalability and memory. Currently, the existing hybrid data optimization and energy-efficient methods either adopt complex computations, resulting in excessive energy cost, or have a fixed and low compression ratio (CR). Moreover, each of them focuses on specific types of data. This means that one may outperform another on text data while performing inferior on image data. Therefore, further research on efficient data optimization and energy-efficient mechanisms is a necessity.

This paper proposes an optimized hybrid approach to reduce transmitted data and enhance energy efficiency in WSN, aiming at conquering the challenges posed by resource-constrained sensor nodes. By integrating mobile computing, edge computing, and cloud computing, it achieves a balance between local processing and centralized storage. With the combination of adaptive Huffman encoding (AHE) and root mean square of the successive difference (RMSSD) threshold, it contributes to a higher CR with optimized energy cost. Its scalability allows for accommodating a growing number of IoT devices and sensor nodes with no sacrifice of performance. Further, one energy-efficient architecture is thus designed for 5G-IoT systems and smart cities, which integrates big data optimization, energy-efficient mechanisms, and green architecture powered by artificial intelligence (AI). Such architecture not only benefits the operation efficiency of smart cities but also ensures sustainability and high energy efficiency by optimizing data flow, managing energy cost, and making real-time decisions, which are all essential for the future deployment of smart cities and IoT in the 5G and 6G eras.

2. Related works

In the fact that sensors create enormous amounts of data transmitted to different IoT layers for processing, storage, and analysis, data compression is a feasible avenue for energy-saving. Kahdim and Manaa [8] adopted Zstandard compression to optimize the sensor data, thus decreasing the network overload and the data volume transmitted from the IoT layer to the fog layer. They achieved the compression efficiency of ~70%, fog processing latency of 6.25 ms, throughput of 85.43 B/ms, and maximum IoT sensor reads of 100. However, the evaluated sensor types and data sizes were small, and no encryption or privacy-preserving mechanisms were integrated. For further enhancement of compression performance, hybrid or adaptive methods are intriguing. Giorgi [9] combined lightweight temporal compression (LTC) with Huffman coding to compress the electrocardiography signal, leading to a reduction of the data volume to be transmitted in WSNs. Giorgi employed a "zero-latency predictive filter" based on differential pulse code modulation for lossy compression, where values that exceed a specific tolerance threshold are picked and transmitted to a centralized node. In the meanwhile, redundant data was further minimized by lossless compression based on an adapted exponential Golomb code and purely reliant on the resolution of the "analog-to-digital converter (ADC)". As high compression efficiency as ~90% was achieved while compromising signal integrity. With adaptability, simplicity, and robustness, it acts as a suitable candidate for real-world healthcare monitoring systems. However, its limitations are also impressive: The assumption of reliable communications with acknowledgment impedes its application in ultra-low-power scenarios; for 8-bit ADCs, the average byte length of compressed data may be larger than that of original data, deteriorating the compression performance; the effectiveness of lossless compression is determined by the statistical distribution of prediction errors. For IoT sensor nodes, Abdulzahra et al. [10] adopted lossy symbolic aggregate approximation (SAX) quantization to reduce the dynamic range of sensor data and lossless Lempel-Ziv-Welch (LZW) compression for further reduction, aiming at minimizing the energy cost during data transmission, with the achieved compression efficiency larger than 87%. However, the data loss caused by SAX quantization is irreversible, making it unsuitable for scenarios requiring exact data reconstruction. Its availability for event-driven or highly dynamic data also remains to be discussed because its optimization is concentrated on periodic data.

Even though such hybrid methods of lossy and lossless compression are beneficial, optimizing the complexity of original data and efficiency of data transfer is challenging. To overcome this limitation, Lu et al. [11] proposed an improved algorithm called data tolerable Fan (DFan) on the basis of simplified Fan data compression. By further integrating DFan with Lempel-Ziv 4 (LZ4) compression, they declined the power consumption, lowered cooling requirements, and ensured sustainable operation. In this method, the data was split into lossy and deviation components. This enhanced data integrity and reduced the risks of eavesdropping. Although

it resulted in a larger CR of >4.0 with a low percentage of root-mean-square difference (PRD), its performance was degraded on datasets with sharp fluctuations. Based on AHE and run length encoding (RLE), Mishra et al. [12] realized the energy cost reduction in WSN for IoT applications and edge computing. They also evaluated the energy efficiency of such a lightweight compression algorithm on low-power microcontrollers like TI MSP430. They demonstrated that data compression before transferring leads to a longer lifetime of WSN, especially in energy-constrained environments. However, it performed superior with periodic data but inferior for random data. With the synergistic superiority of Huffman encoding and LZW, Shrividhiya et al. [13] achieved a CR of 2.417, saving percentage of 57.7%, and compression factor of 0.425. This method is especially promising for electronic communication systems and could be extended to other domains like image processing. However, its performance was only evaluated on general data. For highly structured or multimedia data (e.g., images and audio), it needs further investigation. In order to optimize the performance and sustainability of interconnected systems, El-Henawy et al. [14] developed a hybrid algorithm based on dictionary and Huffman compression, which assigns short codes to letters and patterns that appear more frequently while long codes to those appearing less frequently. The results showed that it obviously outperformed individual Huffman and dictionary-based compression methods, particularly for text data with repeated patterns. Karupusamy et al. [15] combined schizophrenia (SZ) with Huffman coding to compress IoT data, by binding high repetitive symbols with short codes while those repeated less with lengthy codes. Its performance evaluated on a specific physiological dataset indicates that it is well-suited for wearable and edge-based IoT applications, but its generalizability to other domains remains to be unknown.

In summary, to realize an energy-efficient hybrid big data optimization method for 5G-IoT, the following key limitations are lying on the table. First, although edge computing offers benefits such as reduced latency and decreased cloud workload, currently, only a few compression techniques are specifically optimized for deployment on resource-constrained edge devices. Second, lossy compression often accompanies deterioration of data quality, posing another challenge for sensitive applications such as healthcare; hybrid models making a balance between compression efficiency and information loss are lacking. Third, compression approaches capable of dynamically adapting to contextual factors, such as varying environmental conditions and real-time system constraints, are scarce. In addition, most current studies still rely on synthetic or small-scale datasets that fail to capture the complexity and variability inherent in real-world IoT environments. Although machine learning (ML) is increasingly applied for tasks like classification and prediction, its integration with compression processes, particularly in the form of feature-aware or semantic compression, is still in its early stage. Further investigation on how ML algorithms can dynamically guide and optimize compression decisions in real time is also necessary.

3. Proposed method and architecture for 5G-IoT systems and smart cities

3.1. Hybrid data optimization method

Here an improved hybrid data optimization method is proposed, as shown in Fig. 1, where edge computing and cloud computing are synergistically leveraged to achieve a balance between local processing and centralized storage. The localized processing capabilities of sensor and mobile computing significantly decrease the data volume required to be transmitted to the cloud [16]. In the meanwhile, distributing processing tasks across different network layers ensures energy-intensive computations occur where resources are more abundant, contributing to prolonged battery lifetime for sensor nodes. In the proposed approach, a mobile application (App) is developed based on Java to capture, save, and analyze data collected from the built-in sensors on a phone or a tablet, such as pressure, proximity, GPS, light, humidity, and temperature sensors. The Bluetooth or wireless module is also embedded to connect the mobile with the edge device for data transmission. Since the edge, such as a local server, can handle enormous amounts of data efficiently, the collected data is first compressed via the edge. Thus, the data volume to be processed via the data center can be minimized, further contributing to reduced power consumption, decreased transmission latency, and increased network lifetime [17, 18]. In the proposed method, it is assumed that the energy cost by transmission is far larger than that by processing. The original data is first grouped, and the data size is thus reduced. It requires that each new data gathered by a sensor is appended to the end of all preceding sensed data to keep the order of the sensed data unchangeable, which is especially critical in healthcare applications. Thus, sensing and measuring data with a specific sensor will result in data repetition. For example, when tracking the motion of an object

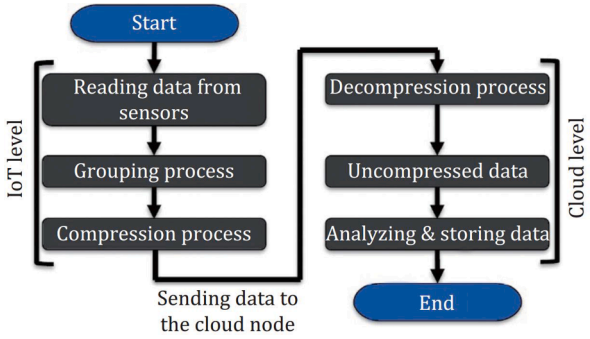


Fig. 1. Flowchart of the proposed method.

placed in front of a given sensor, similar observed values may be recorded multiple times until it moves outside the sensor's range. It is also worthy to note that, for some sensors, such as light sensors, when events change swiftly, only a slight change may occur in the recorded values [8]. This results in a lower CR due to the small number of grouped data. Inspired by the variability estimation of electrocardiogram (ECG) heart rate [19], we use the RMSSD value as a baseline for comparison. It is defined as the mean difference between two consecutive sensor readings of a given sensor and formulated as

$$\mathrm{RMSSD} = \sqrt{\frac{1}{T-1}\sum_{i=1}^{T}\left(s_i - s_{i-1}\right)^2} \tag{1}$$

where T is the total number of collected sensed values, s_i is the new (current) value, and s_{i-1} is the prior value. If the computed absolute difference is less than or equal to RMSSD, the new value is deemed equal to the prior value and aggregated with the prior one; thus, the values with slight changes are compressed and delivered only once. Otherwise, it is saved as a new one and transmitted as a separate value. Obviously, with such grouping method, it is required to first calculate the entire number of subsequent equivalent or approximate values, and then store these values as a single row with the sensed value, representing the whole group of these sensed values.

By combining RMSSD with AHE, an energy-saving and data compression algorithm, named RMSSD-AHE, is proposed, as detailed in Algorithm 1. RMSSD-AHE calls the adaptive Huffman encoding procedure (named AHEC as shown in Algorithm 2) to further reduce the data volume and energy cost. With AHE, the code will be dynamically adjusted to adapt to the frequency of the symbols as they are encountered during data encoding, which is especially useful for real-time data compression or scenarios where the data stream is unknown or subject to change over time [20]. Based on the sibling property, it forms a Huffman tree that can be dynamically modified. As a result, it only requires one pass over the input data, which is especially superior to traditional Huffman coding. In AHC, neither the sender nor the receiver is aware of the statistics of the source sequence before transmission, thus all symbols that have not yet transmitted (NYT) have a weight of 0 in both the transmitter and receiver [12]. As transmission takes place, new nodes are added to the tree, updating the tree to reflect the symbols that have been transmitted. The sender and receiver decide on a predetermined code for each symbol before the transmission starts. Before the usual code of the symbol is transmitted, the NYT node codes are sent. After that, the symbol is given its node and removed from the NYT list. Both the sender and receiver have the same tree structure. Encoding and decoding are always done simultaneously. Nodes are arranged in a specific sequence for the update procedure in the second phase. The NYT node has the lowest node number, while the tree root has the highest number. The numbers are assigned in ascending sequence from lower to higher levels, beginning at the NYT node and moving down the tree. A collection of nodes with the same weight together is called a block.

Algorithm 1. Proposed energy-saving and data compression algorithm (RMSSD-AHE)

```
Initialize an empty list for sensed values (SensedValues);
Initialize an empty adaptive Huffman tree (huffman_tree);
Difs ← 0;
RMSSD ← 0;
SensedValuesIntervals ← 0;
Input SensedValues = collected values from sensor;
Calculate SensedValuesIntervals:
  for each sensed value in the SensedValues do
    Calculate its SensedValues[] interval;
    Append the sensed value interval to SensedValuesIntervals;
  end for each
Calculate RMSSD from SensedValuesIntervals:
Initialize a variable for the sum of squares of differences (sum_of_squares) to 0;
  for each pair of successive sensor value intervals in SensedValuesIntervals do
    Calculate the difference between the pair;
    Square the difference and add it to sum_of_squares;
    Divide sum_of_squares by the number of RR (R-wave peak to R-wave peak) interval pairs;
    Take the square root of the result to get RMSSD;
    Return RMSSD;
  end for each
Compress data:
  for j ← 1 to the length of SensedValues do
    Difs ← Absolute(SensedValues[j] − SensedValues[j−1]);
    if Difs ≤ RMSSD then
      Update Output.LastRow(appearance = appearance + 1);
    else
      Output.Add(SensedValues[i], 1);
    end if
  end for
Call adaptive Huffman encoding procedure (Output)//Algorithm 2.
```

Algorithm 2. Proposed adaptive Huffman encoding procedure (AHEC)

```
Procedure1 AdaptiveHuffmanEncode(input: string):
  if tree is empty
    Initialize tree with a single NYT node;
    Output NYT code;
    Create new leaf nodes for the symbol and NYT;
    Update tree;
    node = findNode(symbol);
  end if
  if node is not found
    Output code for NYT;
    Create new leaf nodes for the symbol and NYT;
    Update tree;
  end if
Output code for the node;
Update tree;
end Procedure1
Procedure2 findNode(symbol):
  if symbol is in the tree
    node = find node containing the symbol;
    Update tree by incrementing the weight of the node;
    while node is not the root
      Find the sibling with the same weight as the node;
      Swap the nodes' positions;
      Update tree;
      Move to the parent of the node;
    end while
  else
    node = find NYT node;
    Create new internal node for NYT and symbol;
    Link the new node to the parent of the NYT node;
    Update tree;
    Return the new internal node;
  end if
end Procedure2
Procedure3 updateTree():
node = root;
  while node is not null
    Find the node with the highest weight among node and its siblings;
    Swap the nodes' positions;
    Update weights;
    Move to the parent of the node.
  end while
end Procedure3
```

AHE is particularly beneficial in applications where the data is sequential and the distribution of symbols changes over time, such as text. It has been demonstrated that AHE performs better than the static one in environments where preprocessing is impossible or impractical [21–23]. It also eliminates the need to transmit a frequency table, which can save space resources especially in smaller or more variable data streams [24]. However, how to manage the reorganization of the tree without excessive computational overhead is still challenging. As the AHE algorithm operates with $O(\log_{10} n)$ complexity for each update, where n is the number of symbols [24], its complexity can be effectively reduced by data compression.

The compressed data is then transmitted to the cloud server from the edge node for decompression, storage, and analysis. The decompression process is shown in Algorithm 3. Then, the cloud server stores the data for further processing and analysis. The proposed method performs all decomposition computations on the cloud side to reduce the energy cost. Fig. 2 illustrates the

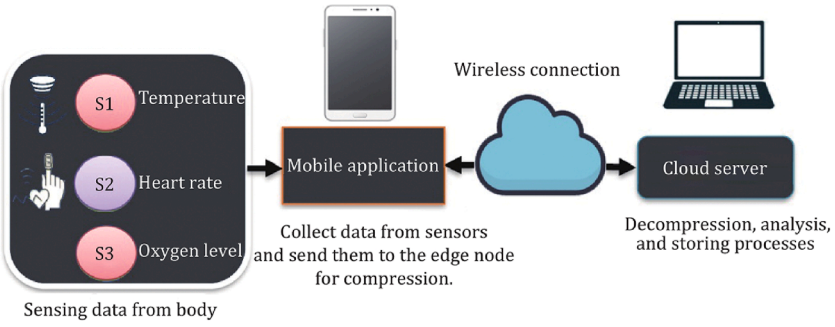


Fig. 2. Implementation of the proposed compression algorithm.

implementation of the proposed data optimization algorithm.

Algorithm 3. Decompression of the proposed RMSSD-AHE

```
1: Procedure1 DecodeBit(bit, output):
2: Traverse the tree according to the input bit (0 for left and 1 for right);
3:   if a leaf node is reached
4:     Output the corresponding symbol;
5:     Update the tree based on the symbol;
6:   else
7:     Continue traversing the tree;
8:   end if
9: end Procedure1
10: Procedure2 DecodeMessage(input):
11:   while there are bits in the input
12:     Decode a bit using DecodeBit procedure.
13:   end while
14: end Procedure2
```

3.2. AI-powered green and energy-efficient architecture

As the digital landscape expands with the proliferation of IoT ecosystems, the evolution of smart cities, and the widespread adoption of cloud computing, the need for green computing is increasingly urgent. By integrating data compression with energy-efficient mechanisms, we propose a novel AI-powered green architecture for 5G-IoT systems and smart cities, as illustrated in Fig. 3. It depicts the interconnected relationships between data compression, energy-efficient mechanisms, and green computing, indicated by the arrows, as well as their impacts on green computing in various technological domains. The proposed architecture, built by the key layers including smart cities and green IoT ecosystems, 5G edge network layer with energy-efficient protocols, data optimization and compression, big data and green cloud computing, and data analysis in cloud and data centers, enables sustainable smart city operations by balancing processing requirements with energy cost through intelligent data management and renewable energy integration.

3.2.1. Bottom/foundation layer: Smart cities and green IoT ecosystems

This layer forms the foundation for various IoT devices and sensors, including smart meters, traffic sensors, waste management, air quality monitoring, smart lighting, water management, energy grid, and smart buildings. These devices collect real-time data while operating with energy-efficient protocols.

The distributed sensing and actuation infrastructure is the foundation of smart cities. Advanced metering infrastructure (AMI) is employed in modern smart meters, where low-power communication protocols such as Zigbee and long-range wide area network (LoRaWAN) are leveraged, thus striking an optimal balance between data granularity and energy cost. With intelligent duty cycling, these devices transmit data only when significant changes occur rather than at fixed intervals, thereby extending the battery lifetime from months to years. Moreover, by incorporating TinyML frameworks to perform preliminary object detection and classification directly on the device, the data volume required to be transmitted is significantly reduced.

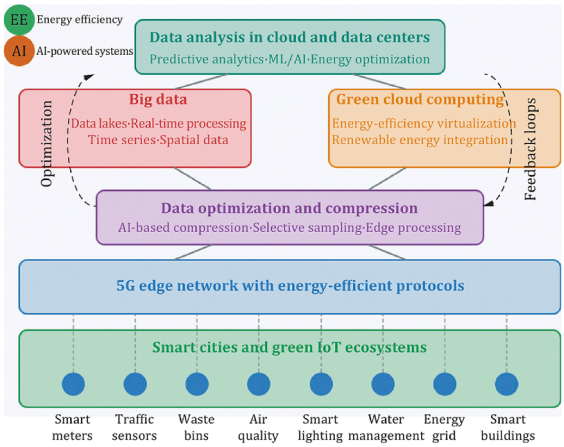


Fig. 3. Proposed AI-powered green and energy-efficient architecture for 5G-IoT systems and sustainable smart cities.

3.2.2. Communication backbone: 5G edge network layer with energy-efficient protocols

This layer provides the communication infrastructure with energy-efficient protocols. It enables low-latency connections between IoT devices and higher processing layers and optimizes data transmission to reduce the energy cost.

Multiple energy-efficient technologies are implemented in the 5G infrastructure that connects these diverse IoT sensors. Network slicing creates virtualized, logically isolated network segments dedicated to specific scenarios, allocating only necessary resources to each application. Critical infrastructure receives dedicated bandwidth, and latency guarantees through URLLC protocols, while mMTC efficiently handles thousands of low-bandwidth sensors per cell. The deployment of multi-access edge computing (MEC) at cell towers brings processing closer to data sources, implementing intelligent data aggregation and filtering before being transmitted to central systems. These edge nodes run specialized workloads optimized for advanced RISC machines (ARM) or reduced instruction set computer-five (RISC-V), offering superior performance-per-watt compared to traditional x86 architectures.

AI-enhanced massive multiple-input multiple-output (MIMO) is also applied to further decrease the energy cost in 5G networks. The transmission patterns are continuously learned, and beamforming configurations are dynamically adjusted, thus minimizing the power consumption while maintaining connectivity. In addition, dynamic time division duplexing (TDD) algorithms are employed to analyze real-time traffic patterns [25] and dynamically adjust uplink/downlink time allocations to match actual usage rather than static configurations.

3.2.3. Efficiency layer: Data optimization and compression layer

As a critical layer between the edge devices and cloud systems, the data optimization and compression layer has a dominant impact on energy cost. At this layer, hybrid data optimization and energy-efficient algorithms proposed in subsection 3.1 are adopted to reduce the data volume, selective sampling is used to minimize unnecessary data transmission, and edge processing is utilized to filter and preprocess data before cloud transmission.

3.2.4. Upper layer: Big data and green cloud computing

This layer comprises two sublayers: Big data and green cloud computing. The former is responsible for managing data lakes, real-time processing, time series, and spatial data analysis, while the latter implements energy-efficient virtualization and integrates energy sources.

1) Storage and processing foundation: Big data

Specialized time series databases like InfluxDB and TimescaleDB are employed to implement automatic down-sampling strategies that preserve long-term trends while reducing storage requirements. This means that high-resolution data for recent periods is maintained, while older data is progressively aggregated to lower resolution based on configurable retention policies.

At this sublayer, distributed storage systems are built on Apache Cassandra to implement geo-distributed replication, thus maintaining data locality to minimize latency while ensuring resilience through geographic distribution. Stream processing frameworks such as Apache Flink provide dynamic parallelism, automatically scaling computational resources based on the volume and complexity of input data. Data lakes are handled by Apache Iceberg to provide efficient partitioning strategies optimized for IoT data patterns, thus significantly accelerating analytical queries.

During the implementation process, hot/warm/cold data tiering based on access patterns is incorporated, automatically migrating less-accessed data to lower-cost storage and achieving seamless access through unified query data lifecycle management systems applied to automatic archiving. With this, even though historical data is moved to archival storage, searchability is maintained through comprehensive metadata.

Lambda architecture is introduced to provide immediate insights for operational decisions while supporting deeper trend analysis for strategic planning. Because this kind of architecture is capable of analyzing current data in real time while analyzing historical data comprehensively.

2) Sustainable processing sublayer: Green cloud computing

Smart cities in the era of 5G and 6G leverage numerous interconnected sensors and devices to collect and analyze data, enabling efficient urban management and resource allocation. However, the operation of these devices often entails substantial energy cost. Green computing can address this limitation by implementing energy-efficient mechanisms at various levels of the smart city infrastructure [26]. From intelligent street lighting systems to optimized transportation networks, these mechanisms reduce the energy cost while enhancing the functionality of smart city services. Thus, by integrating green computing principles with smart cities, a balance between sustainability and technological innovation can be prospective.

This sublayer aims to implement energy-efficient virtualization and integrate renewable energy sources. First, power management through dynamic voltage and frequency scaling (DVFS) is applied to optimize processor performance based on the actual computational load, rather than operating at fixed frequencies. Specialized hardware accelerators are also suggested to provide energy-efficient execution for typical IoT analytics workloads. Typically for FPGAs, 10–50 $\times$ performance improvements can be obtained, compared to general-purpose processors. Second, workload consolidation algorithms are employed to maximize server utilization by intelligently co-locating compatible workloads. This contributes to more efficient operation and realizes server power-down during low-demand periods. Finally, for the best implementation, real-time carbon intensity monitoring is suggested to be integrated with public utility application programming interfaces (APIs). Based on the trend of detected grid carbon intensity, non-critical workloads are scheduled

during periods of lower carbon intensity. Because these battery storage systems integrate with smart charging and discharging algorithms, they will store energy during renewable generation peaks while discharging during high carbon intensity periods.

3.2.5. Top/intelligence layer: Data analysis in cloud and data centers

With its abilities of predictive analytics, ML/AI processing, and energy optimization, this layer facilitates making strategic decisions to improve the overall efficiency of the system. Energy-saving features, including dynamic voltage scaling, task scheduling algorithms, and idle power management, are integrated into this layer. This contributes mobile devices to minimizing the energy cost with no sacrifice of performance or user experience [27]. By leveraging high-performance computing frameworks, particularly the Apache Spark with GPU acceleration, the proposed architecture enables complex analysis across petabyte-scale datasets. Thus, the energy cost can be minimized through workload optimization. Here ML frameworks, including TensorFlow, are employed to implement quantization for efficient inference. The computational requirements are reduced by up to 75% with minimal reduction in accuracy. Visualization platforms like Grafana and Kibana are used to provide decision-makers with intuitive interfaces to complex data. For better performance, digital twin technologies built on gaming engines, like Unity or Unreal Engine, can be incorporated to virtually represent urban environments for scenario planning and simulation. In our implementation, the predictive maintenance model is developed based on XGBoost gradient-boosting frameworks. It is adopted to analyze sensor data and identify possible failure of equipment, enabling pre-emptive intervention before service disruptions occur. In addition, the isolation forests and autoencoder networks are applied for anomaly detection, which can identify unusual patterns across vast sensor networks, flagging potential issues for human investigation.

4. Experimental results and discussion

4.1. Performance evaluation of hybrid data optimization method

To evaluate the performance of the proposed hybrid data optimization method, two cases were compared. The first one adopted the data without compression, while the second one was investigated with the data compressed by the proposed RMSSD-AHEC algorithm.

4.1.1. Evaluation metrics

A total of 5 metrics were taken into consideration to evaluate the proposed method, including CR, throughput, delay (latency), saving percentage, and PRD. Amongst, CR indicates the ratio of the input to the output file size of a compression algorithm, i.e., the source file size before compression to the file size after compression; throughput is used to calculate the maximum data volume that may be transmitted across an Internet connection in a certain period, from source to destination [12]; latency is to quantify the time it takes to transfer data to its destination across a network, which has a significant impact on the network's overall performance; saving percentage measures the compression effect, referring to the ratio of the decreased file size with compression to the original file size; PRD is employed to quantify the percentage variance between the original data and the reconstructed data (the level of accuracy of reconstructed data) [28], as follows:

$$\mathrm{PRD}\ (\%) = 100 \times \sqrt{\frac{\sum_{j=1}^{N-1} \left(D_o(j) - D_r(j)\right)^2}{\sum_{j=1}^{N-1} \left(D_o(j)\right)^2}} \tag{2}$$

where N is the entire number of instances/rows of the dataset, D_r is the reconstructed data (the data returned after decompression), and D_o is the original data. A lower PRD score indicates that the reconstructed data is more accurate.

4.1.2. Results of the first case (without compression)

The "Intel Lab Wireless Sensor Network" dataset collected by University of California[1] was adopted for investigation. This dataset consists of the environmental data (such as humidity, voltage, temperature, and light) collected by 54 sensors in the surrounding lab from February 28 to April 5, 2004, totaling around 2.3 million readings. During experiment, different numbers of IoT sensors (18, 36, and 54) with varying input data sizes (39756 KB, 98972 KB, and 147374 KB) were studied, respectively, and the experimental results are shown in Table 1. The transfer rate was 100 Mbps over N wireless protocols in an optimal situation. The experimental results indicate that the throughput rises as the number of IoT sensors increases.

4.1.3. Results of the second case (with compression)

In this case, the input sensor data was first compressed by the proposed RMSSD-AHEC algorithm. The experimental results summarized in Table 2 show that the proposed algorithm dramatically reduces the data size and network latency by lowering the data volume transmitted from the source (edge node) to the destination (cloud server). The data was then reconstructed based on the generated compressed file. To estimate the accuracy of the reconstructed data, the PRD was calculated by averaging the sum of

[1] University of California (2004). Intel lab data [online]. Available: http://db.csail.mit.edu/labdata/labdata.html (accessed 8 January 2024).

Table 1
Experimental results of the first case study.

Number of IoT sensors	Original data size (MB)	Transfer time (s)	Throughput (MB/s)	Cloud processing latency (s)
18	38.80	3.43	90.50	5.34
36	96.60	6.51	100.56	9.74
54	143.92	11.45	118.71	13.57

Table 2
Experimental results of the second case study.

Number of IoT sensors	Original data size (MB)	Compressed data size (MB)	CR	Transfer time (s)	Throughput (MB/s)	Cloud processing latency (s)
18	38.80	9.46	4.10	0.95	79.66	2.56
36	96.60	26.15	3.69	2.00	104.60	3.81
54	143.92	40.51	3.55	3.97	81.63	4.56

Table 3
Comparison with other related data compression methods.

Reference	Algorithm	Original data size (MB)	Compressed data size (MB)	CR	Saving percentage (%)	PRD (%)
[13]	Huffman based LZW	143.92	51.81	2.78	64.00	11.40×10^{-3}
[8]	Zstandard	143.92	43.17	3.33	70.00	12.47×10^{-3}
[9]	Huffman coding with LTC	143.92	60.45	2.38	58.00	10.34×10^{-3}
[12]	RHE and AHE	143.92	68.47	2.10	52.42	9.36×10^{-3}
[10]	SAX-based LZW	143.92	41.74	3.45	71.00	12.66×10^{-3}
[14]	Huffman coding and dictionary	143.92	41.87	3.44	70.91	12.64×10^{-3}
[11]	DFan with LZ4	143.92	44.62	3.23	69.00	12.30×10^{-3}
[15]	SZ-based Huffman coding	143.92	41.31	3.48	71.30	12.71×10^{-3}
Proposed method	RMSSD-AHE	143.92	40.51	3.55	71.85	12.83×10^{-3}

absolute differences between the original data and the reconstructed data, and the resulted PRD value is only 0.01283%. It demonstrates that the compression performance of the proposed RMSSD-AHEC algorithm is excellent. Moreover, the energy-saving percentage of the proposed method is calculated to be 71.85%, further indicating its feasibility in minimizing energy cost. These results demonstrate that the proposed approach is a superior alternative for IoT resource-limited environments due to its scalability, lower overhead, and less consumption of computational resources.

4.1.4. Comparison with related data compression methods

To further demonstrate the superiority of the proposed hybrid data optimization method, a performance comparison with other related methods was conducted. The corresponding details are shown in Table 3. It is obvious that our method performed superior to others in terms of CR and power consumption while having a comparable and negligible reconstruction error.

4.2. Performance evaluation of AI-powered green architecture for 5G-IoT and smart cities

4.2.1. Simulation methodology

To evaluate the proposed architecture, we utilized a multi-layered simulation framework comprising network simulation tools (NS-3), energy cost models, and data processing frameworks. The simulation environment was configured to mimic a mid-sized urban area (10 km^2) with the following characteristics: 5000 heterogeneous IoT devices distributed across 8 device categories, 15 5G base stations with MEC capabilities, 3 distributed data centers, and an integrated renewable energy (solar/wind) source providing 30%–70% of required energy based on time of day. To comprehensively assess the proposed architecture, the following performance indicators were measured.

- Energy efficiency: Joules per bit transmitted and processed;
- Latency: End-to-end, segmented by the architectural layer;
- Bandwidth utilization: Effective throughput after compression/optimization;
- Processing efficiency: Operations per watt at edge and cloud layers;
- Scalability: Performance degradation with increasing device density;
- Sustainability impact: Estimated CO_2 reduction compared to the baseline.

Two baseline architectures, i.e., traditional architecture and standard 5G architecture, were constructed for comparison. In

Table 4
Latency performance.

Scenario	Latency (ms)			Improvement (%)
	Traditional architecture	Standard 5G architecture	Proposed architecture	
Emergency response	248	87	32	63.2
Traffic management	312	146	52	64.4
Air quality monitoring	453	182	104	42.9
Energy distribution	276	131	58	55.7
Waste management	524	403	215	46.7

traditional architecture, centralized cloud processing with minimal edge computing was adopted. While in the standard 5G architecture, industry-standard 5G was deployed but no green optimization was applied.

4.2.2. System-level performance results

The proposed architecture demonstrated significant energy-savings across all simulated scenarios, owing to the decreased data volume to be transmitted, reduced bandwidth requirement, and enhanced computation efficiency. In the proposed architecture, the data volume was reduced by 61% with adaptive sampling, the required bandwidth was decreased by 73.9% with data compression, and the efficiency was improved by 54.5% with workload-aware scheduling. In other words, the total energy cost of system was reduced by 47.3% compared to the traditional architecture and 26.8% compared to the standard 5G one. The energy-savings varied across architectural layers. For example, 32.1% reduction in energy cost was realized at the IoT device layer, 41.7% reduction at the network infrastructure, and 68.4% reduction at the computing infrastructure.

The latency performance of the proposed architecture was also compared with the traditional and standard 5G architectures under the application scenarios of emergency response, traffic management, air quality monitoring, energy distribution, and waste management. The results are shown in Table 4, where the improvement indicates the latency change compared to the standard 5G architecture. Obviously, significant reduction in the latency was observed across all studied application scenarios, especially in those time-critical applications, such as emergency response and traffic management.

4.2.3. Layer-specific evaluations

To experimentally validate the simulation results, 1000 physical sensors (monitoring the temperature, humidity, motion, and air quality) were deployed in a controlled environment, and the adaptive sampling algorithms mentioned before were implemented in these sensors. The experimental results showed that the battery life was extended by 3.2–4.7 times for different types of sensors, the network traffic was reduced by 58.4% with local preprocessing, and the average response time of sensors for critical events was 122 ms. In addition, as high detection accuracy as 99.8% was achieved despite the reduced sampling.

The energy efficiency of various components within the 5G infrastructure under different load conditions was evaluated with the network simulation. The results demonstrated that network slicing reduced energy cost by 28.3% through optimized resource allocation, AI-optimized MIMO configurations reduced transmission energy by 31.7%, dynamic TDD allocation improved spectrum efficiency by 22.6%, and predictive handover mechanisms reduced signaling overhead by 43.8%. Moreover, the simulation also showed that under peak load (8 a.m.–6 p.m.), the network maintained 99.7% availability and simultaneously only consumed 62.4% power of traditional implementations.

Besides, the compression efficiency, processing overhead, and information loss across various common data types in smart cities were also evaluated. It can be found that the overall required bandwidth was declined by 73.9%. Different reductions in data volume were observed for different data types. For example, the time series sensor data was reduced by 89.3% and video surveillance feeds were reduced by 63.2% with edge processing, while those for voice/audio data and textual/log data were 71.8% and 86.5%, respectively. With regard to the CPU overhead, it averaged 4.7% for edge devices and 2.1% for gateways, however, the information loss after compression was only $<0.3\%$ for critical data.

4.2.4. Scalability analysis

To investigate the system scalability, the deployed IoT devices were gradually increased from 5000 to 50000. Correspondingly, it was observed that the energy cost increased sub-linearly with the power law exponent of 0.72. The latency degradation remained below 15% up to 30000 devices. As the deployed devices further increased, near-linear scalability was provided by additional MEC nodes. More importantly, even during the simulated traffic surge, 99.8% system availability was maintained by automated load balancing.

4.2.5. Impact on environment

To study its impact on the environment, the carbon footprint (CO_2) of the architecture was calculated using a lifecycle assessment approach. The results obtained include 12600 tonnes CO_2 for the embedded carbon infrastructure, 1870 tonnes CO_2 annually (with current energy mix) for operational emissions, 27300 tonnes CO_2 annually for the carbon reduction from optimized city operations, and 25430 tonnes CO_2 annually for net carbon reduction. The corresponding carbon payback period is 6 months.

5. Limitations and prospects

Although inspiring simulation results have been achieved, there still remain several challenges.

- In terms of security, a careful balance between robust protection and energy efficiency is required. This leads to increased adoption of lightweight cryptography algorithms like PRESENT and CLEFIA for constrained devices.
- With regard to deployment, API-first design principles enable ecosystem development around core infrastructure, accelerating innovation through third-party applications. Adherence to open standards like FIWARE and oneM2M ensures interoperability between systems from different vendors and across municipal boundaries. However, real-world deployments may face integration challenges with legacy systems. For example, the availability of renewable energy varies with regions due to the different weather; requirements of regulatory compliance may limit the application of some optimization approaches.

In the future, quantum-resilient encryption will be possibly incorporated into this architecture. Edge-cloud continuum approaches will further blur the distinction between device, edge, and cloud processing, enabling seamless workload migration based on energy availability and processing requirements. As AI capabilities continue advancing, increasing autonomy may shift more decision-making to distributed systems, with human operators transitioning toward exception handling and strategic oversight rather than routine management. In addition, future work may also include extended pilot deployments in diverse urban environments, enhancing fault tolerance under extreme conditions, developing standardized APIs for cross-city interoperability, as well as quantifying quality-of-life improvements from architecture implementation.

6. Conclusions

To solve the limitations of transmission efficiency and energy cost in 5G-IoT applications and smart cities, an optimized hybrid data optimization approach was first proposed in this paper by combining the RMSSD threshold with AHE. It yielded high performance with a large CR of 3.55, reduced power consumption of 71.85%, and extremely small reconstruction error of 0.01283%, respectively. It is demonstrated superior to the existing compression algorithms due to its scalability, lower overhead, and less computation of power resources. By leveraging mobile, edge, and cloud computing simultaneously, the performance and energy of interconnected systems were optimized with the achieved balance between local processing and centralized storage, providing a potential candidate for the 5G-IoT resource-limited environment. Based on the proposed hybrid approach, a novel AI-powered green and energy-efficient architecture for 5G-IoT systems and sustainable smart cities was further proposed, aiming at the current limitations in infrastructure, security, and interoperability. With 47.3% reduction in energy cost, 54.6% lower latency, and 25430 tonnes CO_2 annual reduction, it is demonstrated that the proposed architecture not only can improve the efficiency of smart cities but also can ensure its sustainability. Its implementation provides a viable pathway for cities to improve the service, reduce the cost, and meet the sustainability goal simultaneously.

CRediT authorship contribution statement

Ihab Nassra: Writing–original draft, Visualization, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization, Validation. **Juan V. Capella:** Writing–review & editing, Supervision, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] M. Agiwal, N. Saxena, A. Roy, Towards connected living: 5G enabled Internet of things (IoT), IETE Tech. Rev. 36 (2) (2019) 190–202.
[2] W. Li, Y.-B. Chai, F. Khan, et al., A comprehensive survey on machine learning-based big data analytics for IoT-enabled smart healthcare system, Mobile Netw. Appl. 26 (1) (2021) 234–252.
[3] A.P. Johnson, S. Patranabis, R.S. Chakraborty, D. Mukhopadhyay, Remote dynamic partial reconfiguration: a threat to Internet-of-things and embedded security applications, Microprocess. Microsy. 52 (2017) 131–144.
[4] K.-Y. Cao, Y.-F. Liu, G.-J. Meng, Q.-M. Sun, An overview on edge computing research, IEEE Access 8 (2020) 85714–85728.
[5] Y. Ai, M.-G. Peng, K.-C. Zhang, Edge computing technologies for Internet of things: a primer, Digit. Commun. Netw. 4 (2) (2018) 77–86.
[6] Atta-ur-Rahman, S. Dash, M. Ahmad, T. Iqbal, Mobile cloud computing: a green perspective, in: S.K. Udgata, S. Sethi, S.N. Srirama (Eds.), Intelligent Systems: Proceedings of ICMIB 2020, Springer, Singapore, 2021, pp. 523–533.
[7] S. Bebortta, S.S. Tripathy, U.M. Modibbo, I. Ali, An optimal fog-cloud offloading framework for big data optimization in heterogeneous IoT networks, Decision Analytics Journal 8 (2023) 100295.
[8] A.N. Kahdim, M.E. Manaa, Design an efficient Internet of things data compression for healthcare applications, Bull. Electr. Eng. Inf. 11 (3) (2022) 1678–1686.
[9] G. Giorgi, A combined approach for real-time data compression in wireless body sensor networks, IEEE Sens. J. 17 (18) (2017) 6129–6135.
[10] S.A. Abdulzahra, A.K.M. Al-Qurabat, A.K. Idrees, Data reduction based on compression technique for big data in IoT, in: Proc. of 2020 Intl. Conf. on Emerging Smart Computing and Informatics, Pune, India, 2020, pp. 103–108.
[11] S.-F. Lu, Q.-H. Xia, X.-L. Tang, X.-Y. Zhang, Y.-P. Lu, J.-K. She, A reliable data compression scheme in sensor-cloud systems based on edge computing, IEEE Access 9 (2021) 49007–49015.
[12] M. Mishra, G.S. Gupta, X. Gui, Investigation of energy cost of data compression algorithms in WSN for IoT applications, Sensors 22 (19) (2022) 7685.

[13] G. Shrividhiya, K.S. Srujana, S.N. Kashyap, C. Gururaj, Robust data compression algorithm utilizing LZW framework based on Huffman technique, in: Proc. of 2021 Intl. Conf. on Emerging Smart Computing and Informatics, Pune, India, 2021, pp. 234–237.
[14] I.M. El-Henawy, E.R. Mohamed, N.A. Lashin, A hybrid technique for data compression, International Journal of Digital Content Technology and Its Applications 9 (2) (2015) 11–20.
[15] S. Karupusamy, J. Refonaa, S. Sankaran, P. Dahiya, M.A. Haq, A. Kumar, Effective energy usage and data compression approach using data mining algorithms for IoT data, Expert Syst. 40 (4) (2023) e12997.
[16] M.F. Khan, R.K. Dwivedi, R. Kumar, Energy efficient data transmission in sensor cloud: a review, in: Proc. of 2019 3rd Intl. Conf. on Trends in Electronics and Informatics, Tirunelveli, India, 2019, pp. 308–313.
[17] T.N. Gia, Q.-Q. Li, J.P. Queralta, H. Tenhunen, Z. Zou, T. Westerlund, Lossless compression techniques in edge computing for mission-critical applications in the IoT, in: Proc. of 2019 Twelfth Intl. Conf. on Mobile Computing and Ubiquitous Network, Kathmandu, Nepal, 2019, pp. 1–2.
[18] M. Bagherian, S. Chehade, B. Whitney, A. Passian, Classical and quantum compression for edge computing: the ubiquitous data dimensionality reduction, Computing 105 (7) (2023) 1419–1465.
[19] D. Nikolova, P. Mihaylova, A. Manolova, P. Georgieva, ECG-based human emotion recognition across multiple subjects, in: Proc. of the 4th EAI Intl. Conf. on Future Access Enablers for Ubiquitous and Intelligent Infrastructures, Sofia, Bulgaria, 2019, pp. 25–36.
[20] J.S. Vitter, Design and analysis of dynamic Huffman codes, J. ACM 34 (4) (1987) 825–845.
[21] P.R.R. Kumar, M. Prabhakar, An integrated approach for lossless image compression using CLAHE, two-channel encoding and adaptive arithmetic coding, SN Computer Science 5 (2024) 523.
[22] S. Arrabi, J. Lach, Adaptive lossless compression in wireless body sensor networks, in: Proc. of the Fourth Intl. Conf. on Body Area Networks, Los Angeles, USA, 2009, pp. 19:1–8.
[23] Y. Zhou, M.S. Mulekar, P. Nerellapalli, Adaptive spam filtering using dynamic feature spaces, Int. J. Artif. Intell. Tool. 16 (4) (2007) 627–646.
[24] A. Moffat, Huffman coding, ACM Comput. Surv. 52 (4) (2020) 85.
[25] F.S. Samidi, N.A.M. Radzi, W.S.H.M.W. Ahmad, F. Abdullah, M.Z. Jamaludin, A. Ismail, 5G new radio: dynamic time division duplex radio resource management approaches, IEEE Access 9 (2021) 113850–113865.
[26] N.I. Sarkar, S. Gul, Green computing and Internet of things for smart cities: technologies, challenges, and implementation, in: B. Balusamy, N. Chilamkurti, S. Kadry (Eds.), Green Computing in Smart Cities: Simulation and Techniques, Springer, Cham, Germany, 2021, pp. 35–50.
[27] S. Ki, G. Byun, K. Cho, H. Bahn, Co-optimizing CPU voltage, memory placement, and task offloading for energy-efficient mobile systems, IEEE Internet Things 10 (10) (2023) 9177–9192.
[28] C.K. Jha, M.H. Kolekar, Electrocardiogram data compression using DCT based discrete orthogonal Stockwell transform, Biomed. Signal Process. 46 (2018) 174–181.



Ihab Nassra received the B.S. and M.S. degrees in computer science from Arab American University (AAUP), Jenin, Palestine in 2001 and 2018, respectively, where he was the Head of the Quality Assurance and Information Security Unit from 2018 to 2020. He is currently pursuing the Ph.D. degree in computer science at Polytechnic University of Valencia (UPV), Valencia, Spain. He is also a lecturer with the Department of Computer Science, AAUP and a researcher at the Medical Sciences Center, AAUP and the Institute of Information and Communication Technologies, UPV. His research interests include wireless sensor networks, big data, distributed systems design, information security, artificial intelligence, machine learning, medical informatics, and Internet of things.



Juan V. Capella received the B.E. degree in computer science and the M.S. degree in computer engineering from UPV in 1998 and 2002, respectively, where he is currently pursuing the Ph.D. degree. He was the Head of the Hewlett-Packard (HP) Institute, UPV from 2013 to 2015. Now he is also an associate professor with UPV and a member of the Smart Electronic and Networks Research Group (SENS), Internationalizing Italian Academia in Africa (ITACA). His research interests include distributed systems design, wireless and underwater sensor networks, and Internet of things.